Security architecture: trusted controls over agent-proposed actions

Current implementation | Complete action binding, deterministic policy checks, single-action approval, and pre-execution revalidation

TRUSTED CONTROL PLANE / roles use distinct credentials

A Administrator / capability registry

Register tool ID, actions, path roots, effects and parameters.
Sign capability; only the latest grant is active.
Independent operator file: deny actions / require parameter bounds. Default: no business bans.

B Trusted user channel / intent anchoring

Intent LLM proposes symbolic rules and expected effects. Server owns the signed profile.
New input replaces policy and increments epoch; old tickets become stale. Taint persists.

C Independent approver / operator setup

Approve exact pending action only; never grant permanent rule expansion.
Startup requires state outside the workspace, provisioned keys, mandatory checks, and no mock evidence.

capability

rules + effects

RUNTIME AUTHORIZATION / serialized gateway mutations; one process / worker

1 Agent + adapter

SDK / LangChain / BrowserGym / pi
Bind defaults; freeze complete JSON arguments.
Submit tool, action, session and capability ID.

Input boundary

Executor role required.
Strict request schemas.
Server session must exist.
Unknown or failed responses stop execution.

2 Evidence + provenance

Allowlisted attached browser: live capture.
A11y + OCR + optional Tesseract / grounding.
Fuse regions; optional time consistency.
Client evidence cannot establish trust.

3 Signed action evidence

Bind full action hash, target/type, session, epoch and expiry.
Include sources, verifier versions, roles, region and crop hash.
Ed25519 signature.

4 Policy engine + behavior alignment

Hard checks: business rules, capability parameters, active grant, paths, deny rules, taint and evidence.
Intent rules: exact tool match and finite sensitive-field constraints.
Behavior: compare extracted effects against intent and capability restrictions.
Uncertainty escalates or blocks (Figure 2).

5 Gateway decision controls

Changed tool implementation: CONFIRM.
Approval may override CONFIRM only; budget exhaustion becomes BLOCK.

BLOCK / deny

Stop; no execution ticket.

CONFIRM / pending

Independent approval issues one-use ticket.
Re-authorize the identical action; repeat hard checks.

ALLOW / execution ticket

Consume once before execution.
Recheck policy, state and evidence; then invoke the trusted tool.

Approval + consumption protocol: Figure 3

Shared state and tamper-evident audit

Session policy / epoch / effects / taint; pending, approval and execution tickets.
Decision, approval and consumption records use a signed hash chain.
Raw action bodies and screenshots are omitted from the default audit log.

Explicit implementation boundary

Gateway authorizes; it is not an OS sandbox. The executor and tool code are trusted.
Production blocks host shell / arbitrary process tools. GUI demo tools are previews.
Parameter bounds are enforced; implementation hashes are not runtime attestation.

Solid arrows: request / decision flow. Dashed arrows: trusted control inputs. Detailed ordering and conditional paths are expanded in Figures 2-3.

Policy semantics: hard boundaries, conditional evidence, and effect alignment

Order follows policy.engine.evaluate() then policy.authorize.authorize_action(); P6 can hard-block a prior ALLOW or CONFIRM

(a) ENGINE DECISION ORDER

01 Independent business rules + capability bounds

Operator deny/require rules run first; default empty. Enforce capability parameter bounds. Then require policy or valid rules, declared action and YAML capability requirements.

02 File boundary + argument consistency

Resolve absolute paths; enforce exact or directory grants. Empty envelope denies. Reject traversal / foreign paths / Windows alternate streams; reject conflicting target aliases.

03 Explicit deny + direct sensitive reference

Matching deny rule: BLOCK. A sink directly referencing a known high-risk object: BLOCK. These checks precede soft approval returns.

04 Evidence certificate, when required / supplied

Verify signature, signed vs outer fields, full action hash and expiry. Visual actions with unknown or conflicting evidence: BLOCK.

05 Intent, finite parameter constraints + uncertainty

No rules: sink / high-risk BLOCK, otherwise CONFIRM. Outside allow rule or insufficient sensitive constraints: CONFIRM. Derived taint / untrusted visual evidence: CONFIRM.

06 Final engine checks

Shell requires a non-dangerous command analysis, otherwise CONFIRM (production already blocks host shell). Remaining checks pass: engine ALLOW.

07 Behavior alignment (P6), after engine

Preserve engine BLOCK. With effect schema + profile, violations become BLOCK. Schema present but profile absent: downgrade engine ALLOW to CONFIRM.

Hard BLOCK is never converted to ALLOW by approval.

(b) POLICY COMPONENTS

Symbolic least privilege

Rules = {tool, allow / deny, predicate}; validated before matching.
DSL: true, and/or/not, eq/ne, le/ge, in/not_in, glob, contains.
Sensitive values require positive finite eq/in constraints on every accepted branch. AND combines constrained fields; OR keeps only common fields.
Raw parameter semantics are preserved: no blanket lowercase or URL decoding.

Expected effects vs extracted effects

Expected: intent targets, recipients, data types, max_items and forbidden effects; capability-denied effects are added to the forbidden set.
Actual: deterministic extraction from declared effect schema + action parameters.
Checks: tool, scoped URL/path targets, recipients, data types, recipient-count limit and forbidden effects. Sensitive unknown effects: BLOCK.
Scope lists are conditional; max_items currently counts recipients, not all operations.

Visual evidence path

A11y has priority: matching perceptual evidence supports it; disagreement yields conflict. A structured source alone can establish trust.
Otherwise: two agreeing source types establish trust; one is untrusted; none is unknown. Region agreement = intersection / smaller area (default 0.5).
Optional previous capture: instability downgrades trust. Attached mode uses a fresh capture without previous-frame comparison.

Two-level taint (P5)

Sources: declared high-risk data / credentials. Sinks: network, sharing, persistence.
Scan nested object values and array elements: direct references BLOCK; other dirty sinks CONFIRM. Depth / size / unsupported-data failures BLOCK.
Mark source taint conservatively at ticket consumption; retain it across user turns.

What these checks do not establish

Provenance does not authorize a task. Intent alignment does not establish request safety. Extracted effects are not observed execution.
Taint scans JSON values, not encoded content or object keys. Parameter bounds are per call, not cumulative budgets.

Conditional execution is explicit: optional verifier backends may be unavailable; effect alignment requires both a declared effect schema and an effect profile.

Authorization lifecycle: approval, one-time consumption, and state revocation

ALLOW is permission to request execution consumption; the signed evidence certificate itself is not an execution token

(a) SINGLE-ACTION APPROVAL AND EXECUTION

1 /authorize

Executor submits full bound request.
All hard checks run.
BLOCK: stop.
ALLOW: issue execution ticket.
CONFIRM: create pending token.

2 /confirm

Independent approver accepts / rejects.
Pop pending token; verify expiry (300 s default), epoch and context hash.
Accept: issue one-use approval ticket. Reject / stale: no grant.

3 /authorize + approval ticket

Pop approval ticket; verify identical session, tool and complete action.
Re-run hard checks.
Only a resulting CONFIRM may be converted to ALLOW.

4 Execution ticket

Server-held random token; 30 s TTL.
Bind action/tool/session, epoch, policy/taint/effects context hash, evidence hash and original request.

Direct ALLOW skips human approval

5 /consume: pop first, then revalidate

Executor role; serialized one-time ticket consumption.
Check TTL, epoch, context hash and complete binding.
Re-authorize stored request with current capability / policy / taint and newly computed evidence.
Require ALLOW and unchanged evidence hash; otherwise refuse execution.

6 State update + audit

If tool is a high-risk source, mark session taint before returning success. Failed reads may over-taint.
Append signed consumption event.
Replay of consumed ticket fails. SDK proceeds only after a successful response.

7 Trusted executor invokes tool

Execute the frozen arguments checked by the gateway.
External boundary: process/file/network permissions must prevent alternate access paths.
No atomic OS-level coupling to /consume; mutable resources still need executor-side protection.

(b) REVOCATION, AUDIT AND DEPLOYMENT GUARDS

Session / tool transitions

New intent: increment epoch, replace rules, reset effects and confirmation budget; retain taint.
Close session: clear policy, effects, taint and all tickets.
New registration invalidates old capability. Changed implementation converts an ALLOW to CONFIRM.

Approval and resource bounds

Default budget: 10 confirmations per epoch; exceeding it yields BLOCK.
Ticket maps: 10,000-entry guard each. Anchored sessions: 1,000-entry guard.
Business policy: startup snapshot, audit hash, restart to reload. One worker.

Audit and credential separation

Ed25519-signed hash chain for recorded decisions, approvals and consumption. Invalid chain stops append.
Four distinct credentials (at least 32 characters). Agent must not hold privileged tokens. Protect gateway code, keys and state with OS permissions.

IMPLEMENTATION SCOPE / DO NOT INTERPRET AS ADDITIONAL ENFORCEMENT

Single trusted deployment domain; user/tenant-to-session authorization is not implemented. Host shell and declared arbitrary process execution are blocked in production.
Capability implementation_hash is not a measurement of the executing binary. Parameter constraints enforce a documented subset: types, values, bounds, nested fields and array items.
Hash chains need external checkpoints to detect suffix deletion. Derived taint may be approved; no complete cross-session or transformed-content lineage is claimed.
pi browser/click/type demo tools are authorization previews. R-Judge policy-assisted judgment is an evaluation path, not part of the runtime authorization chain.

Based on the current repository implementation. All numeric limits shown are defaults; deployment configuration may change configurable limits.